Project Report

RYNEX

A Flex-Based Interactive Domain Specific Language in C++

Course Title: Compiler Design Lab

Course Code: CSE 352

Department: CSE

Submitted to:

Md. Tasnin Tanvir

Lecturer of CSE, University of Information Technology and Sciences

Submitted by:

Name: Md. Raiyan

ID: 0432320005101101

Section: 6B1

Semester: Spring 2026

1. Introduction

The **RYNEX** Programming Language is a custom-designed, interactive Domain Specific Language (DSL) developed as part of the Compiler Design Lab course (CSE 352). **RYNEX** is built using Flex (Fast Lexical Analyzer), a well-known tool for generating lexical analyzers in C, and is implemented entirely within a single Flex source file (raiyan.l) that compiles to a standalone executable.

RYNEX replaces standard C mathematical function calls with uniquely coined keywords that follow a consistent naming convention — all ending in the suffixes 'ixx', 'vall', or 'xx'. This approach encourages students to think creatively about language design while simultaneously reinforcing their understanding of core programming concepts such as arithmetic operations, logical comparisons, mathematical functions, and lexical analysis using Flex.

The project consists of a single Flex source file (raiyan.l) which, when processed by flex and compiled with gcc, produces a fully interactive **RYNEX** interpreter. All 27 functions have been implemented and verified to produce correct results. **RYNEX** also features an interactive numbered menu system, allowing users to select functions by number and enter inputs one at a time — a key feature that distinguishes it from simple batch-mode calculators.

2. Objectives

- To design a custom Domain Specific Language (**RYNEX**) with unique function naming conventions using the Flex lexer generator.
- To implement 27 standard mathematical and logical operations using **RYNEX** keywords in C.
- To demonstrate all implemented functions through a working interactive interpreter (raiyan.l).
- To verify that all proposed functions produce correct and accurate output.
- To build an interactive numbered menu system so users can call functions by number and enter arguments interactively.
- To gain hands-on experience in lexical analysis, compiler front-end design, and C programming.

3. RYNEX Language Overview

RYNEX is implemented using Flex, the industry-standard tool for building lexical analyzers. It does not require a separate parser or interpreter runtime — the Flex scanner itself recognizes each **RYNEX** command token and immediately executes the corresponding C action. The naming convention for **RYNEX** functions is as follows:

Suffix	Category	Examples
-ixx	Core arithmetic / power / sqrt	Addixx, Subixx, Multiixx, Divixx, Modixx, Powerix, Sqrtixx
-vall	Value-based / math / trig / round	Absvall, Avgvall, Factvall, Logvall, Sinvall, Roundvall
-xx	Compare / analysis	Comparexx, Isequall, Notequall, Isgreaterr, Islesss

A key feature of **RYNEX** is its dual input mode: users may either type the full function call directly (e.g., Addixx(10,20)) or press the function's number from the help menu and enter arguments interactively (a = 10, b = 20). The program runs continuously until the user presses Q to quit.

4. Proposal vs. Implementation Analysis

The following table compares every function against the actual implementation in raiyan.l. Verification was performed by compiling and running with sample inputs (a = 10, b = 5).

#	Standard Function	RYNEX Keyword	Description	Category	Status
1	Add / Sum	Addixx(a,b)	Adds two numbers	Basic Math	OK
2	Subtract	Subixx(a,b)	Subtracts two numbers	Basic Math	OK
3	Multiply	Multixx(a,b)	Multiplies two numbers	Basic Math	OK
4	Divide	Divixx(a,b)	Divides two numbers	Basic Math	OK
5	Modulus	Modixx(a,b)	Finds remainder	Basic Math	OK
6	Power	Powerix(a,b)	Raises to power	Adv. Math	OK
7	Square Root	Sqrtixx(n)	Finds square root	Adv. Math	OK
8	Absolute	Absvall(n)	Returns absolute value	Adv. Math	OK
9	Maximum	Maxvall(a,b)	Finds maximum value	Comparison	OK
10	Minimum	Minvall(a,b)	Finds minimum value	Comparison	OK
11	Average	Avgvall(a,b)	Calculates average of two numbers	Statistics	OK
12	Increment	Incvall(n)	Increases value by 1	Utility	OK
13	Decrement	Decvall(n)	Decreases value by 1	Utility	OK
14	Factorial	Factvall(n)	Calculates factorial	Adv. Math	OK

15	Logarithm	Logvall(n)	Calculates log base 10	Adv. Math	**OK**
16	Sine	Sinvall(n)	Finds sine value (degrees)	Trigonometry	**OK**
17	Cosine	Cosvall(n)	Finds cosine value (degrees)	Trigonometry	**OK**
18	Tangent	Tanvall(n)	Finds tangent value (degrees)	Trigonometry	**OK**
19	Random Number	Randvall()	Generates random number (0-99)	Utility	**OK**
20	Round	Roundvall(x)	Rounds a decimal number	Rounding	**OK**
21	Floor	Floorvall(x)	Rounds down to nearest integer	Rounding	**OK**
22	Ceiling	Ceilvall(x)	Rounds up to nearest integer	Rounding	**OK**
23	Compare	Comparexx(a,b)	Compares two values, shows greater	Comparison	**OK**
24	Equal Check	Isequall(a,b)	Checks if two values are equal	Comparison	**OK**
25	Not Equal	Notequall(a,b)	Checks if two values are not equal	Comparison	**OK**
26	Greater Than	Isgreaterr(a,b)	Checks if a is greater than b	Comparison	**OK**
27	Less Than	Islesss(a,b)	Checks if a is less than b	Comparison	**OK**

Result: All 27 functions proposed have been correctly implemented in raiyan.l and verified to produce accurate output. There are no missing or extra functions.

5. Implementation Details

5.1 File Structure

File	Purpose
raiyan.l	Single Flex source file — contains all 27 **RYNEX** function rules, the interactive menu, the prompt loop, and the main() entry point.
lex.yy.c	Auto-generated C file produced by running 'flex raiyan.l'. Never edited manually; regenerated on every flex run.

5.2 Function Categories

Basic Arithmetic (5 functions): Addixx, Subixx, Multixx, Divixx, Modixx — core four operations plus modulus. Divixx includes a division-by-zero guard that prints 'Math Error: Division by zero'.

Advanced Math (4 functions): Powerix, Sqrtixx, Factvall, Logvall — power, square root, factorial, and logarithm base 10. Logvall uses log10() from math.h.

Trigonometry (3 functions): Sinvall, Cosvall, Tanvall — input is taken in degrees and converted to radians internally using the formula: angle * 3.1416 / 180.

Comparison (6 functions): Maxvall, Minvall, Isequall, Notequall, Isgreaterr, Islesss — compare two integer values and return True/False or print the greater value.

Statistics and Utility (4 functions): Avgvall (average of two numbers), Absvall (absolute value), Incvall (increment by 1), Decvall (decrement by 1).

Rounding (3 functions): Roundvall, Floorvall, Ceilvall — accept decimal (double) input and wrap the standard C round(), floor(), and ceil() functions from math.h.

Random and Compare (2 functions): Randvall() generates a random integer from 0 to 99 (seeded with time(NULL) at startup). Comparexx prints which of the two values is greater, or 'Equal' if they match.

5.3 Error Handling

RYNEX includes built-in error handling for unsafe operations:

- Divixx: Prints 'Math Error: Division by zero' and skips the result if the divisor is zero.
- Invalid input: Any unrecognized token triggers the catch-all rule, printing 'Invalid Syntax: ...' followed by a reminder to press H for help.
- Whitespace and newlines are silently ignored by the rule [\t\n]+ ; — without this, every Enter keypress would trigger an error.

5.4 Interactive Features

RYNEX introduces three interactive UX improvements over the original design:

- Numbered Help Menu (H): Pressing H displays all 27 functions with serial numbers in a formatted box. HELP() also works.
- Number Shortcut: Typing a number (1–27) selects that function by name, then prompts for only the required inputs (a and b for two-argument functions, n for single-argument, x for decimal).
- Persistent Prompt: After every operation, 'Enter Command:' is printed and the program waits for the next input. The session only ends when Q is pressed.

6. Build and Run Instructions

The following commands are used to compile and run **RYNEX** on any system with Flex and GCC installed:

Step	Command
Generate C from Flex source	**flex raiyan.l**
Compile generated C file	**gcc lex.yy.c -o rynex -lm**
Run the **RYNEX** interpreter	**./rynex**

The -lm flag links the C math library (libm), required for pow(), sqrt(), log10(), sin(), cos(), tan(), round(), floor(), and ceil().

7. Sample Output

The program was compiled using gcc and executed with inputs a = 10, b = 5. The verified output for all 27 function calls is shown below.

Function Call	Output	Verification
Addixx(10,5)	Result = 15	Correct
Subixx(10,5)	Result = 5	Correct
Multixx(10,5)	Result = 50	Correct
Divixx(10,5)	Result = 2	Correct
Modixx(10,5)	Result = 0	Correct
Powerix(2,8)	Result = 256	Correct
Sqrtixx(144)	Result = 12.00	Correct
Absvall(-10)	Result = 10	Correct
Maxvall(10,5)	Result = 10	Correct
Minvall(10,5)	Result = 5	Correct
Avgvall(10,5)	Result = 7.50	Correct
Incvall(10)	Result = 11	Correct
Decvall(10)	Result = 9	Correct
Factvall(5)	Result = 120	Correct
Logvall(100)	Result = 2.0000	Correct
Sinvall(90)	Result = 1.0000	Correct
Cosvall(0)	Result = 1.0000	Correct
Tanvall(45)	Result = 1.0000	Correct
Randvall()	Result = 47	Correct (varies)
Roundvall(4.6)	Result = 5	Correct
Floorvall(4.9)	Result = 4	Correct
Ceilvall(4.2)	Result = 5	Correct

Comparexx(10,5)	10 is Greater	Correct
Isequall(5,5)	True	Correct
Notequall(10,5)	True	Correct
Isgreaterr(10,5)	True	Correct
Islesss(10,5)	False	Correct

8. Relevance to Bangladesh Education System

This project aligns closely with the objectives of the Bangladesh National Education Policy 2010 and the vision of Digital Bangladesh, which emphasize the integration of Information and Communication Technology (ICT) into higher education.

In Bangladesh's university-level Computer Science and Engineering (CSE) programs — particularly under the curriculum framework guided by the University Grants Commission (UGC) — courses like Compiler Design and Compiler Lab are core requirements. These courses are designed to help students understand how high-level programming languages are translated into executable form, which is fundamental to software engineering.

The **RYNEX** project directly supports these goals in the following ways:

Practical Flex and Lexical Analysis Knowledge: By building a complete DSL using Flex, students gain direct hands-on insight into lexical analysis, token recognition, pattern matching with regular expressions, and action execution — all core topics in the Compiler Design syllabus.

Encourages Creativity: Bangladeshi engineering education often emphasizes rote learning. Projects like **RYNEX** encourage students to design something original with their own naming conventions and interactive features.

Strengthens C Programming: C remains the primary language used with Flex. Building a Flex-based interpreter deepens understanding of C standard libraries (math.h, stdio.h, stdlib.h) and memory management.

Supports Skill Development: The project builds skills directly valued in Bangladesh's growing software industry, including Dhaka's expanding tech hub community.

Assessment Alignment: The structured proposal → implementation → report workflow mirrors the project assessment pattern used in public and private universities across Bangladesh under semester-based grading systems.

9. Conclusion

The **RYNEX** Programming Language project has been successfully completed. All 27 functions outlined in the project have been implemented in the Flex source file (raiyan.l) and verified to produce correct results.

The project demonstrates a strong understanding of lexical analysis using Flex, C standard math libraries, error handling, interactive program design, and custom language development — all of which are core competencies in the Compiler Design Lab course.

RYNEX goes beyond a basic calculator by introducing a numbered interactive menu, function-name-based selection feedback, and a persistent command loop — features that make it genuinely usable as a terminal-based mathematical tool. It serves as a proof-of-

concept that a meaningful, fully functional custom programming language can be built using only Flex and standard C, making it an excellent academic exercise for students studying compiler construction in Bangladesh and beyond.

10. References

1. Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2006). Compilers: Principles, Techniques, and Tools (2nd ed.). Pearson Education.
2. Levine, J., Mason, T., & Brown, D. (2009). lex & yacc (2nd ed.). O'Reilly Media.
3. Kernighan, B. W., & Ritchie, D. M. (1988). The C Programming Language (2nd ed.). Prentice Hall.
4. cppreference.com — C Standard Library Reference. https://en.cppreference.com
5. The Flex Manual — GNU Project. https://westes.github.io/flex/manual/
6. University Grants Commission (UGC) Bangladesh — CSE Curriculum Framework. http://www.ugc.gov.bd
7. Ministry of Education, Bangladesh — National Education Policy 2010. http://www.moedu.gov.bd
8. UITS Course Curriculum, CSE 352: Compiler Design Lab, Spring 2026.